INFRAESTRUCTURA CLOUD

Diseño de una plataforma cloud privada basada en OpenStack y Kubernetes

para la ejecución de modelos de inteligencia artificial generativa

Presentado por:

Kevin Jeovany Castellanos Garay CG13082
Daniel Jesús Servellón Ramos SR17033
Alex Alfonso Munto Uceda MU13002
Melvin Bladimir Suárez SS09023

Docente asesor: Ing. Julio Damián Morales Ayala

Ciudad Universitaria,10 noviembre de 2026



UNIVERSIDAD DE EL SALVADOR
ESCUELA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

ESTRUCTURA DE LA PRESENTACIÓN

Contenido de la presentación

1. Problema	Recursos dispersos y sin administración común
2. Objetivos	Un objetivo general y cinco específicos
3. Diseño y desarrollo	Arquitectura de cinco capas y su construcción
4. Demostración	Recorrido en vivo de 10 minutos

Problema: recursos repartidos sin una capa común

PLANTEAMIENTO DEL PROBLEMA

Determinar si es técnicamente viable una nube privada OpenStack, con Kubernetes como plataforma, que ejecute un servicio de IA generativa, y su evaluación técnica.

PREGUNTA DE INVESTIGACIÓN

¿En qué medida es técnicamente viable una nube privada OpenStack con nodos distribuidos, unidos por WireGuard, que ejecute un servicio de prueba virtual de prendas?

5 Equipos físicos en sitios distintos

0 Direcciones IP públicas ni puertos abiertos

1 GPU NVIDIA RTX 3060 de 12 GBz

Objetivos

OBJETIVO GENERAL

Diseñar y evaluar técnicamente una nube privada basada en OpenStack para ejecutar aplicaciones de IA con aceleración por GPU, con FASHN VTON v1.5 y un clasificador de prendas como casos de estudio, y determinar su viabilidad técnica.

OE1

Identificar requerimientos y capacidades de los equipos

OE2

Diseñar la arquitectura de nube privada con WireGuard

OE3

Implementar OpenStack, la conectividad y la asignaciòn de la GPU

OE4

Integrar VTON y el descriptor de prendas como casos de estudio

OE5

Evaluar rendimiento, monitoreo y pruebas de carga

Arquitectura de cinco capas y monitoreo transversal

Arquitectura por capas del prototipo (cada capa se apoya en la inferior)



Haz clic en la figura para ampliarla

SaaS — VTON Studio, API de VTON y descriptor

PaaS — Kubernetes con Magnum

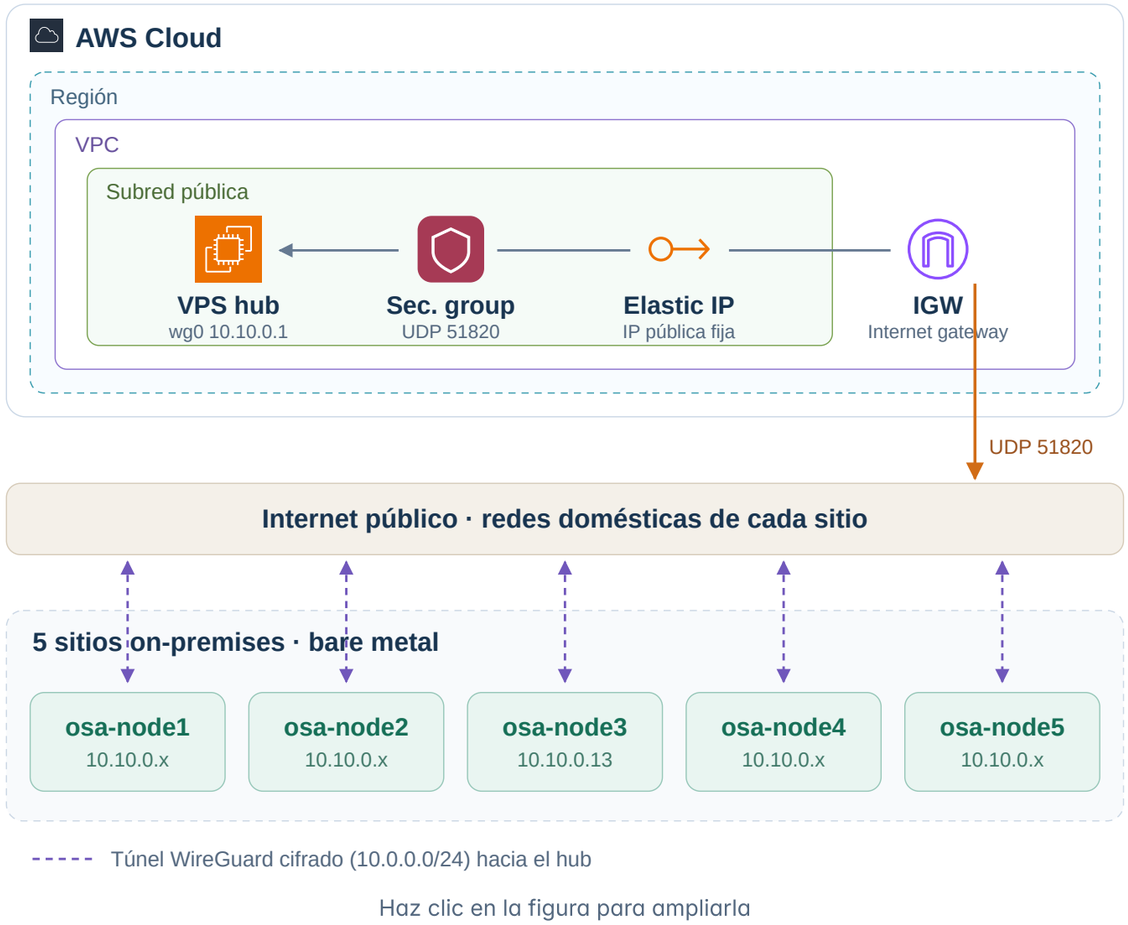
IaaS — OpenStack-Ansible 33.0.1

Red — VPN WireGuard con hub en Amazon EC2

Física — 5 equipos bare metal, 1 con GPU

Cada capa se apoya en la inferior y se verificó antes de construir la siguiente.

Capa física y de red: cinco sitios unidos por WireGuard



Haz clic en la figura para ampliarla

Hub en Amazon EC2

Solo los equipos inician la conexión: funciona tras NAT.

Latencia de 70 a 140 ms

Por salto entre sitios; la MTU de la red privada se fijó en 1370 B.

Roles por capacidad

Controlador, almacenamiento, cómputo y un nodo con GPU.

IaaS: OpenStack administra cómputo, red y almacenamiento



Haz clic en la figura para ampliarla

33.0.1

OpenStack-Ansible

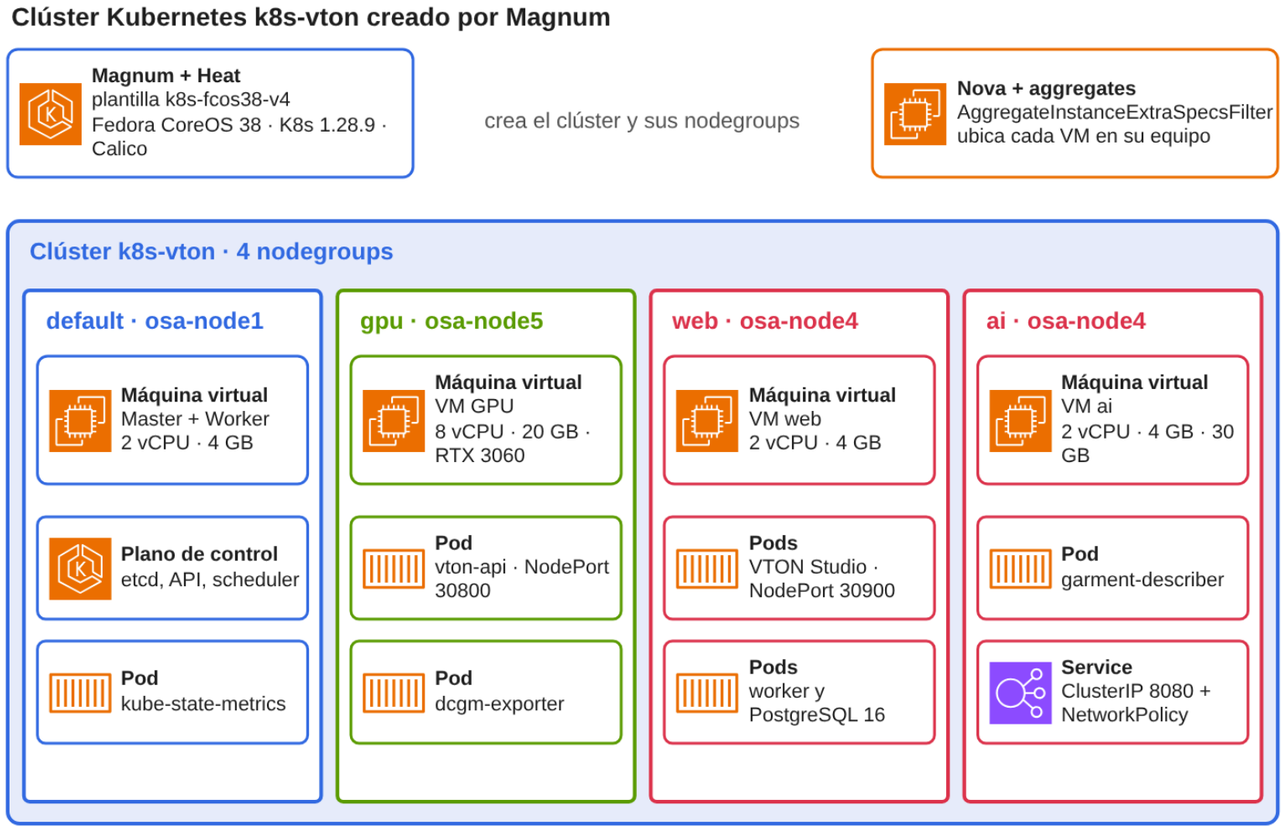
6

servicios: Nova, Neutron, Glance, Cinder, Heat y Magnum

3

hipervisores en estado up

PaaS: Kubernetes con Magnum y la GPU en un pod



Haz clic en la figura para ampliarla

Clúster k8s-vton

CREATE_COMPLETE y HEALTHY, con cuatro nodegroups.

GPU por PCI passthrough

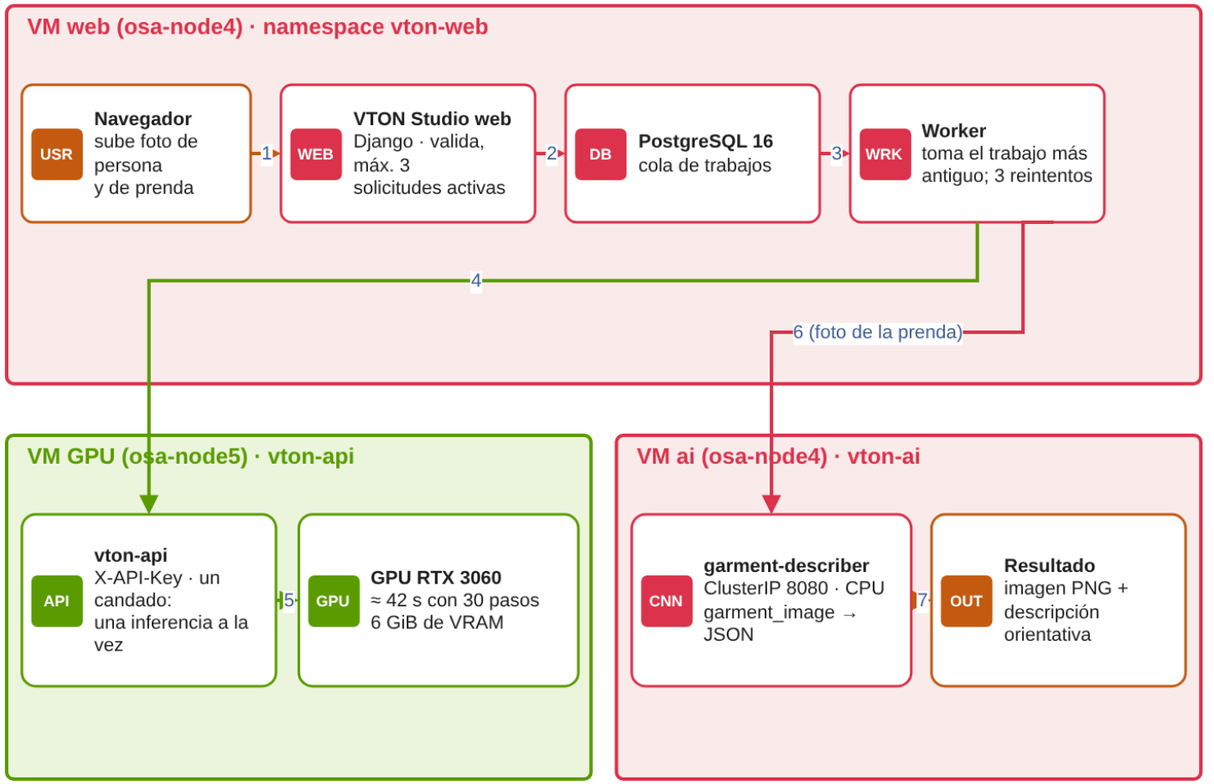
La RTX 3060 se entrega completa a una VM y desde allí a un pod con el device plugin de NVIDIA.

Ubicación controlada

Nova coloca cada VM en el equipo previsto con host aggregates.

SaaS: prueba virtual de prendas con una cola de solicitudes

Flujo del caso de estudio: del navegador a la GPU y de vuelta



Haz clic en la figura para ampliarla

Una sola inferencia a la vez

La GPU atiende un trabajo; los demás esperan en la cola de PostgreSQL.

Límites por usuario

Máximo 3 solicitudes activas por persona; la API exige clave.

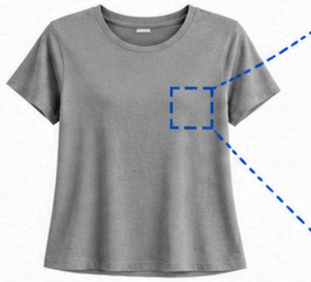
Descriptor de prendas

El worker consulta al descriptor y la imagen se entrega igual si este falla.

Funcionamiento de una red convolucional

Convolución y extracción de características en MobileNetV2-Small

Prenda en escala de grises



Parche 6×6 extraído

220	218	213	205	198	192
221	219	214	206	199	193
218	216	211	203	197	191
210	208	205	198	192	186
205	203	199	193	188	182
200	198	194	189	184	179

Filtro 3×3

−1	−1	−1
−1	8	−1
−1	−1	−1

Convolución (deslizamiento del filtro)

220	218	213	205	198	192
221	219	214	206	199	193
218	216	211	203	197	191
210	208	205	198	192	186
205	203	199	193	188	182
200	198	194	189	184	179

(−1 × 220) + (−1 × 218) + (−1 × 213) + (−1 × 221) + (8 × 219) + (−1 × 214) + (−1 × 218) + (−1 × 216) + (−1 × 211)

= 19

Mapa de características

19	18	16	14
20	18	15	13
17	15	11	9
14	12	9	7

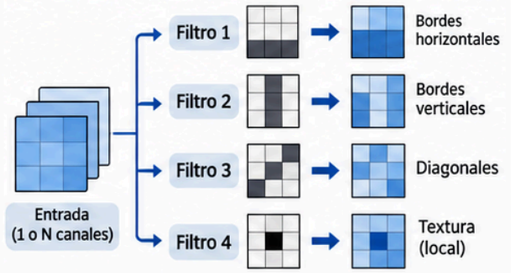
bordes / textura

Separación en canales de color

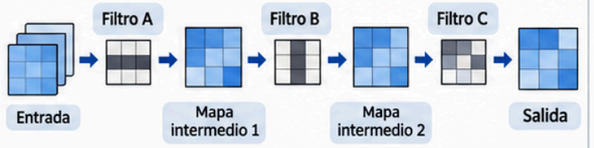


Prenda (RGB)

Múltiples filtros independientes



Múltiples filtros en cadena



En MobileNetV2-Small, estas operaciones permiten detectar formas, bordes, colores y patrones de las prendas.

De fotografías a un dataset etiquetado

Curación, anotación con IA generativa y estructuración de los datos para MobileNetV3-Small

01 · RECOLECCIÓN

Fuentes de imágenes

Prendas procedentes de tres conjuntos de datos.



Clothing Dataset · Clothing Dataset Small
Mendeley Data

Dataset seleccionados por sus licencias abiertas para entrenamiento.

02 · CONTROL DE CALIDAD

Depuración

Se descartan imágenes duplicadas o visualmente similares.



Después de la limpieza:

14.427
imágenes seleccionadas para el dataset

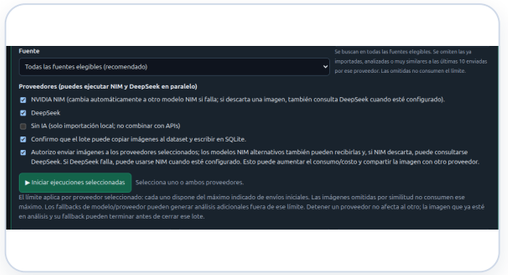
Se creó un script de Python y una interfaz gráfica para identificar repetidas y se validaron visualmente para descartarlas.

03 · ANOTACIÓN AUTOMÁTICA

Inteligencia artificial

Modelos externos extraen características visuales de las prendas.

NVIDIA NIM **DeepSeek**



Captura del panel desarrollado para el proyecto.

Tipo **Color** **Estilo**

La IA genera **etiquetas**; no entrena todavía MobileNet.

04 · CONSOLIDACIÓN

Dataset en SQLite

Cada imagen se relaciona con sus atributos y etiquetas.

DB **Base estructurada**

Tipo	Color	Estilo
Camiseta	Azul	Casual
Vestido	Rosa	Formal
Pantalón	Índigo	Casual

Ejemplo de estructura. Los valores mostrados son ilustrativos.

Dataset listo
para entrenamiento supervisado

Salida: fotografías + etiquetas + base de datos SQLite.

Idea central: dataset depurado y anotado para entrenamiento.

Fuente: informe final · §§ 3.2.5 y 5.5

Entrenamiento, publicación y ejecución en nube

El modelo se entrena fuera de la nube; Kubernetes únicamente ejecuta la inferencia en CPU

01 FUERA DE LA NUBE PRIVADA

Entrenamiento de MobileNetV3-Small

Aprendizaje por transferencia y ajuste fino · salidas independientes para tipo, color y estilo.



Imagen **224 × 224** | Batch **64** | LR **0,001** | Fine-tune LR **0,0001** | Warmup **3** | Mejor época **23**

02 RECURSOS PUBLICADOS Y DESPLEGADOS

GitHub → Cinder → Kubernetes

Se publicaron los pesos y los recursos del dataset; el servicio carga el checkpoint desde un volumen.

GitHub
Código · SQLite
Checkpoint best.pt

→

Cinder
Volumen vton-ai-models
Peso best.pt · 3,8 MB

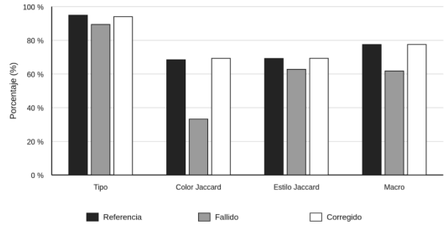
→

Kubernetes
garment-describer
FastAPI · CPU

✓ **Nuevos pesos: copiar al volumen y reiniciar el pod, sin reconstruir la imagen.**

RESULTADOS DEL MODELO FINAL · evaluación controlada con 1.424 fotografías no vistas

Criterio R11: macro ≥ 70 %



94,03 %
Exactitud en tipo

69,28 %
Jaccard de color

69,29 %
Jaccard de estilo

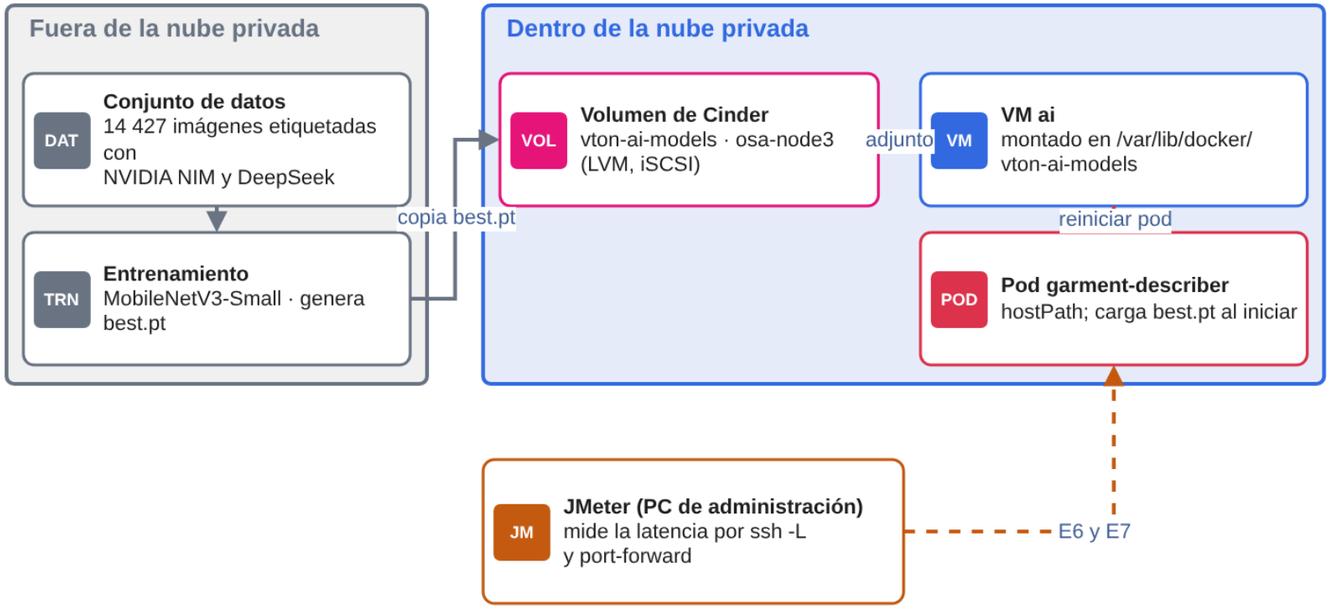
77,53 %
Promedio macro · cumple R11

Entrenamiento corregido: 28/30 épocas; mejor época 23.

Inferencia: 43,6 ms de media en servidor (E6).

Descriptor de prendas: MobileNetV3-Small en CPU

Ciclo de vida de los pesos del descriptor, del entrenamiento al pod



Un archivo nuevo de pesos llega al volumen y basta reiniciar el pod; no se reconstruye la imagen.

Haz clic en la figura para ampliarla

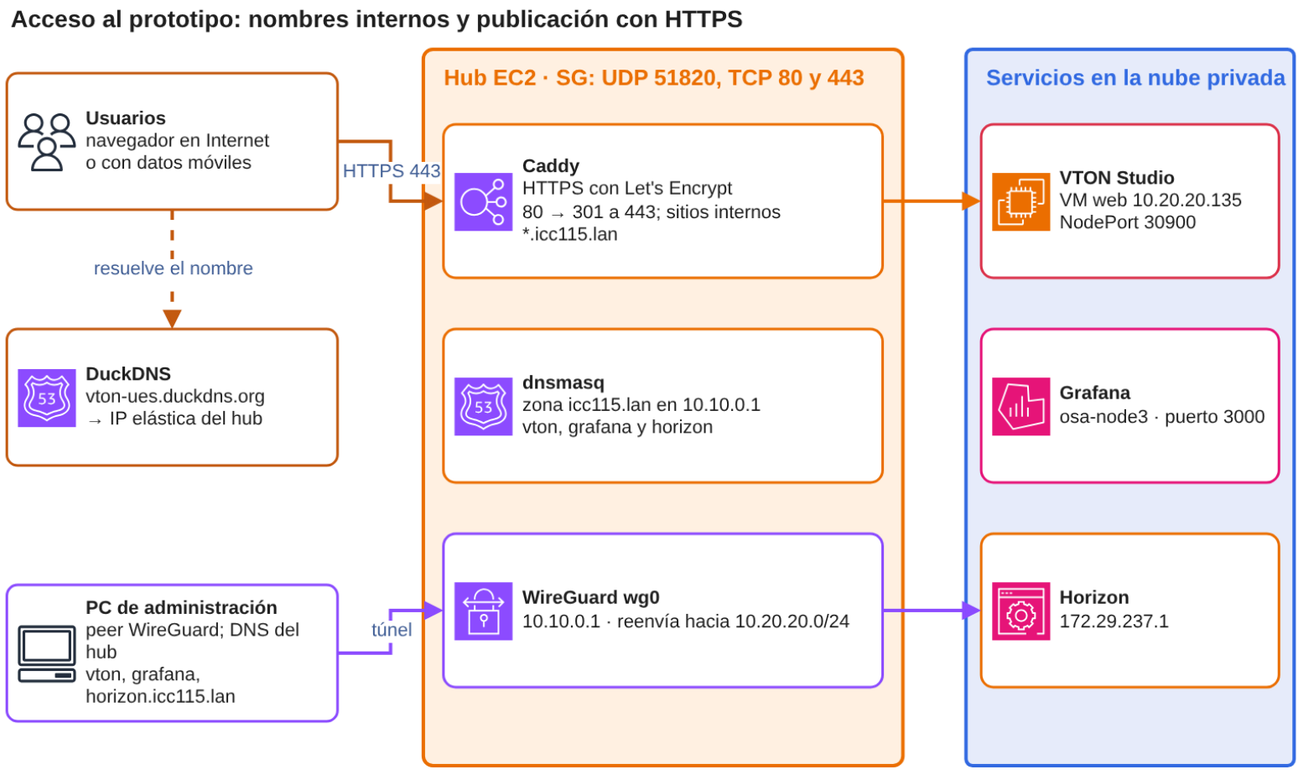
77,5 %

promedio macro de tipo, color y estilo sobre el conjunto de prueba de 1 424 imágenes (criterio R11: ≥ 70 %)

- Entrenado fuera de la nube; solo la inferencia corre en el clúster.
- Los pesos se cargan desde un volumen de Cinder, sin reconstruir la imagen.
- En una prueba funcional con 18 fotos reales: 75,0 % en tipo y 81,2 % en color.

El entrenamiento se hizo en un equipo aparte; la nube solo despliega el resultado.

Acceso con HTTPS y monitoreo de la plataforma



Los servicios internos no se publican: solo VTON Studio sale a Internet, con certificado emitido por Let's Encrypt en el hub.

Haz clic en la figura para ampliarla

HTTPS público

Solo VTON Studio sale a Internet, con certificado de Let's Encrypt que se renueva solo.

Nombres internos

Zona icc115.lan en el hub; Grafana y Horizon solo se alcanzan por la VPN.

Monitoreo

Prometheus y Grafana con 14 targets, incluida la GPU.

4 · DEMOSTRACIÓN

Recorrido en vivo · 10 minutos

1 — 1 min

Acceso público

Abrir el sitio con HTTPS

2 — 3 min

Prueba virtual

Subir persona y prenda; ver la cola y el resultado

3 — 1 min

Descriptor

Tipo, color y estilo de la prenda

4 — 2 min

Grafana

Uso de GPU y VRAM mientras corre el trabajo

5 — 2 min

Nube

Horizon y kubectl: nodos y pods

6 — 1 min

Pruebas

Resultados de JMeter

Resultados: FASHN VTON en la GPU de la nube

20,7 s
inferencia en frío
(8 pasos de difusión)

11,3 s
inferencia en caliente
(8 pasos de difusión)

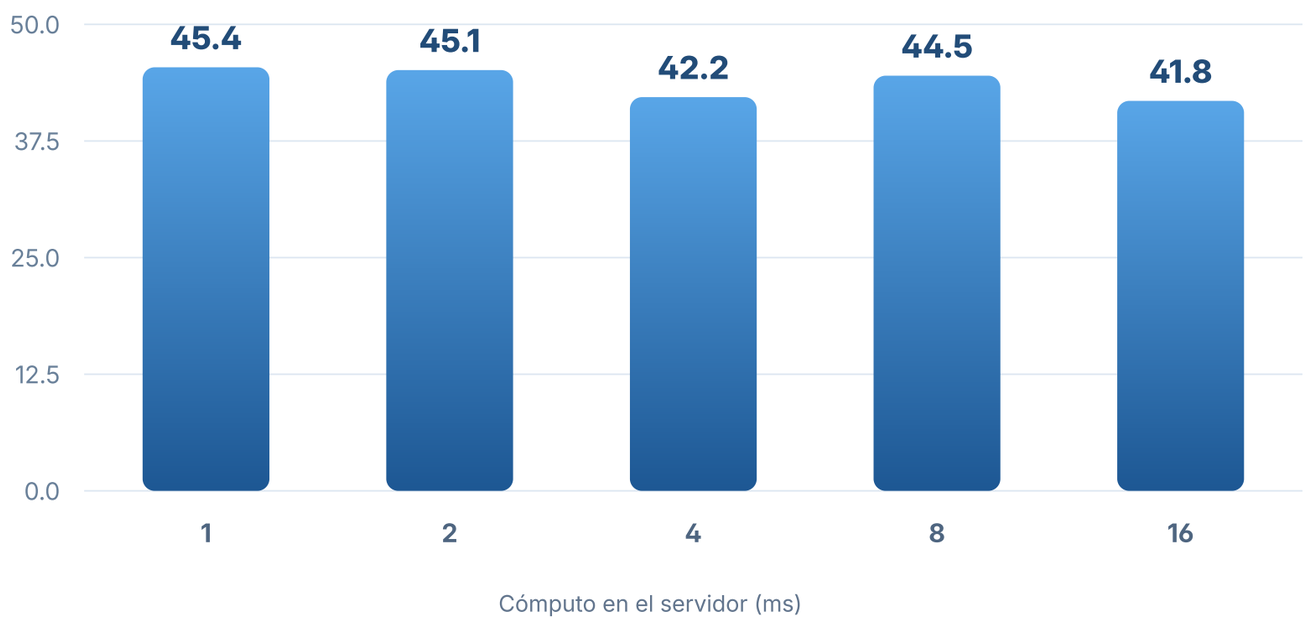
40–50 s
flujo web completo
(30 pasos)

6,1 GiB
de VRAM de 12 GiB;
una réplica más no cabe

Qué implica la cola de una sola solicitud en la GPU

- La GPU llegó al 100 % de uso, estable, con 6256 MiB.
- Con tres solicitudes simultáneas se estiman unos 126 s hasta la última (3 × 42 s); es una estimación, no una medición.
- Los tiempos de 8 y 30 pasos no son comparables entre sí.

Resultados: descriptor medido con JMeter



El tiempo de cómputo se mantiene plano de 1 a 16 usuarios.

43,6 ms media en el servidor en E6 (1 usuario, 97 muestras), con 0 errores

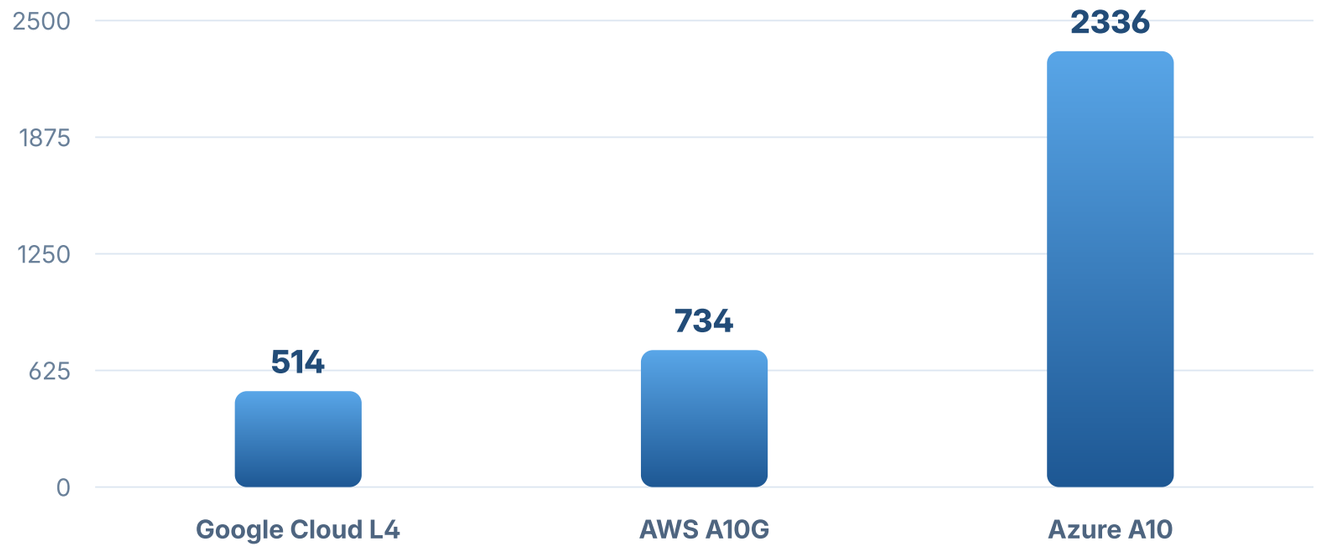
0 respuestas 4xx/5xx de la aplicación; el pod no se reinició

Capacidad máxima no determinada

Los errores de E7 (25 a 64 %) son de transporte del túnel de pruebas, con 10 % de pérdida en el enlace doméstico. Falta repetir desde una red estable.

Factibilidad y costos evitados

Alquiler de una instancia con GPU en nube pública (USD por mes)



USD por mes (730 h)

Tarifas de lista bajo demanda consultadas en noviembre de 2026; las instancias no son idénticas a la RTX 3060 del equipo propio.

Sin licencias

OpenStack, Kubernetes y WireGuard son software libre.

Gasto externo recurrente

Solo la instancia EC2 del concentrador.

Alcance

Factible en un escenario controlado; no es un estudio económico integral.

Limitaciones observadas y trabajo futuro

Condiciones observadas

- El concentrador y el controlador son puntos únicos de falla.
- Los servicios internos aún no usan TLS; solo el borde público lo usa.
- Una sola GPU atiende una solicitud a la vez.
- La recuperación depende de personas en sitio.

Mejora posible

- Repetir E7 desde una red estable.
- Validar el try-on con 2, 3 y 5 usuarios.
- TLS interno, rotación de claves y respaldos.
- Segundo concentrador y más GPU si crece la demanda.
- Reentrenar el descriptor con fotos de uso real.

Conclusiones

Es viable

Cinco equipos dispersos se integraron en una nube privada con OpenStack, Kubernetes y una GPU compartida.

Soporta IA generativa

VTON v1.5 corre en la GPU de la nube y el descriptor en CPU, con pesos entrenados fuera de ella.

Con condiciones

Es un prototipo funcional en escenario controlado, no apto para producción sin TLS interno, alta disponibilidad y respaldos.

Aporte reproducible

Procedimientos, configuraciones y fallas documentados para repetir la construcción.



Gracias

¿Preguntas?

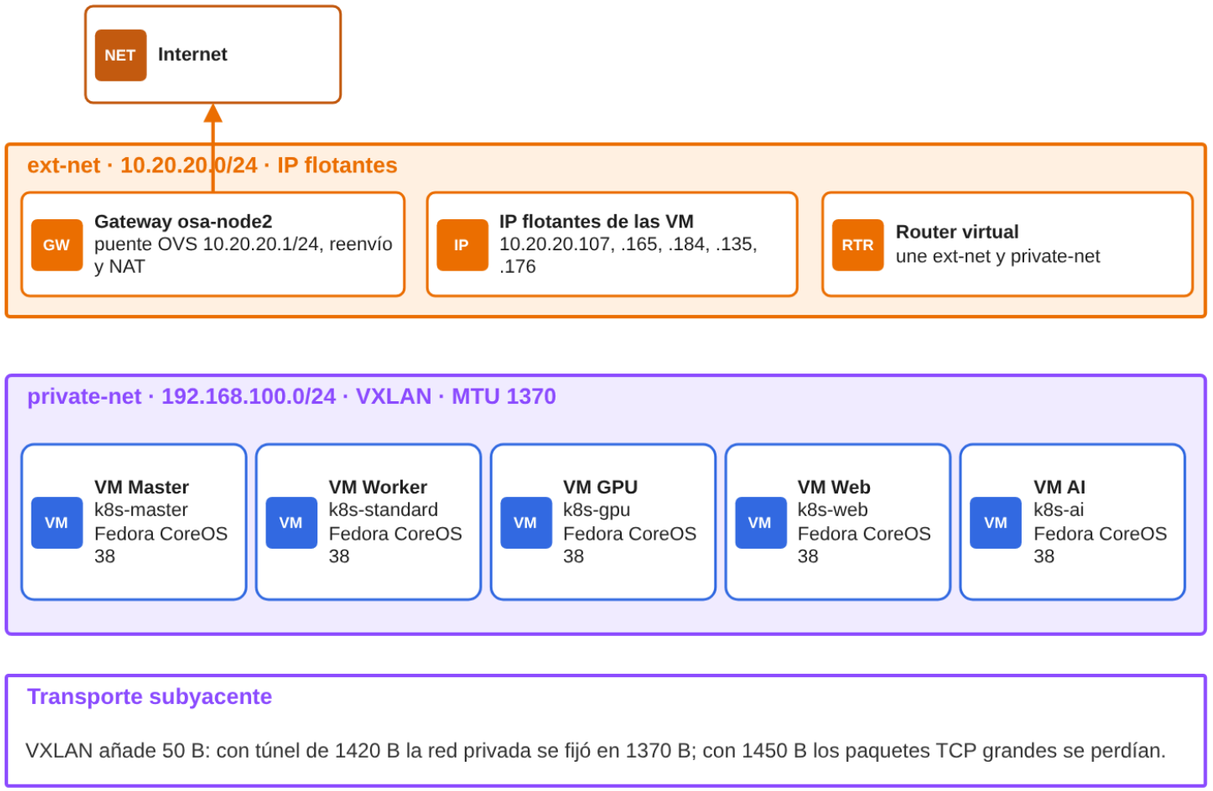
Kevin Jeovany Castellanos Garay · Daniel Jesús Servellón Ramos · Alex Alfonso Munto Uceda · Melvin Bladimir Suárez

Universidad de El Salvador · Escuela de Ingeniería de Sistemas Informáticos

DIAPOSITIVA DE RESPALDO

Respaldo: red virtual de la nube

Red virtual de la nube (Neutron): de las VM a Internet



Haz clic en la figura para ampliarla

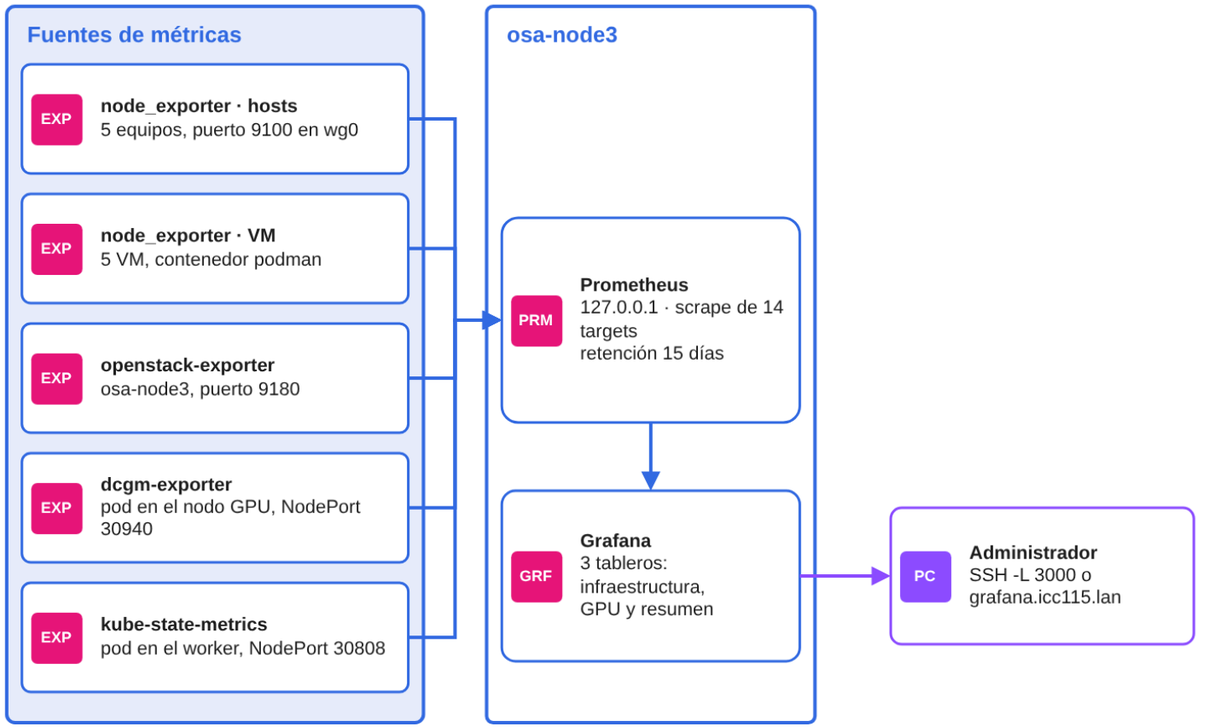
ANEXO TÉCNICO

Neutron crea la red privada con VXLAN; el gateway y el NAT de ext-net viven en osa-node2. La MTU de 1370 B evita pérdida de paquetes por el encapsulamiento.

DIAPOSITIVA DE RESPALDO

Respaldo: arquitectura de monitoreo

Monitoreo del prototipo (fuera de OpenStack y de Kubernetes)



Se descartó Telemetry de OpenStack: con passthrough no ve la GPU y obligaba a modificar Nova, RabbitMQ y Galera.

Haz clic en la figura para ampliarla

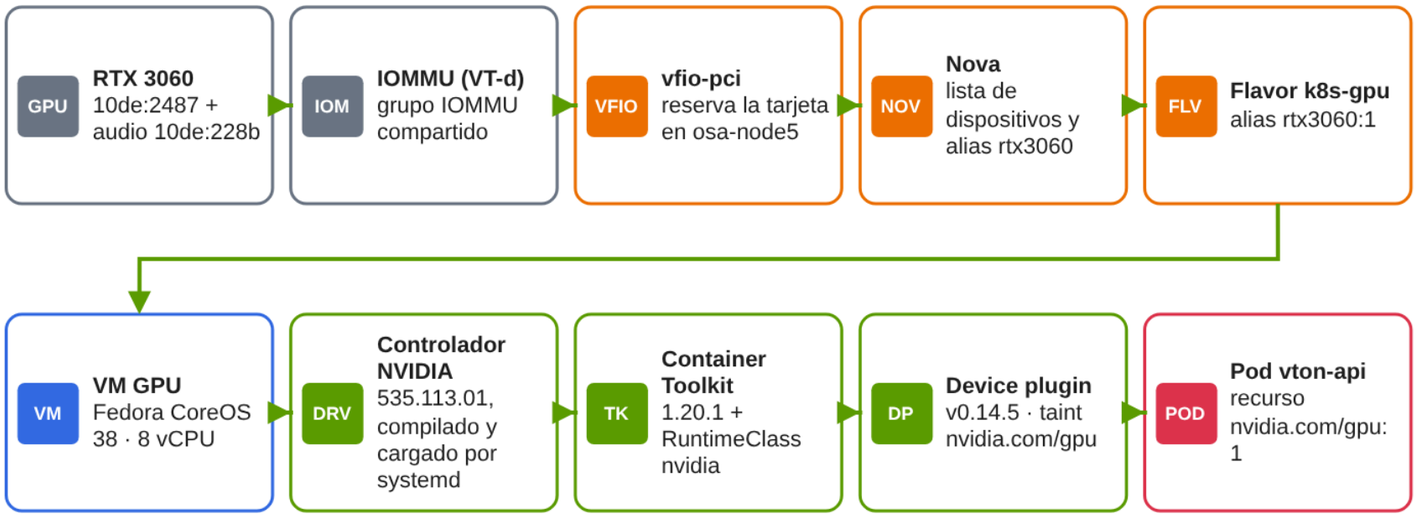
ANEXO TÉCNICO

Prometheus consulta 14 targets y Grafana muestra tres tableros. Se descartó Telemetry de OpenStack porque con passthrough no ve la GPU.

DIAPOSITIVA DE RESPALDO

Respaldo: ruta de la GPU

Ruta de la GPU: del hardware al pod



Mientras la VM posee la tarjeta ningún otro consumidor puede usarla, y OpenStack deja de ver su uso: la GPU se mide con dcgm-exporter dentro de la VM.

Haz clic en la figura para ampliarla

ANEXO TÉCNICO

La GPU pasa del hardware al módulo vfio, a la máquina virtual, al driver, al toolkit, al device plugin y al pod.